Lab Exercise: Income Tax Management System Using Spring Boot, MySQL, JWT, and React

Lab Objective

1. Implement a **full-stack CRUD application** for managing taxpayers and their income.
2. Apply **Spring Boot, Spring Data JPA, and MySQL** for backend development.
3. Secure the backend APIs with **JWT-based authentication**.
4. Implement a **React frontend** using **Formik, Yup, and Axios** for form handling, validation, and API calls.
5. Perform CRUD operations (Create, Read, Update, Delete) via both **Postman** and **React frontend**.
6. Practice project structuring, validations, and authentication in a lab setting.

Prerequisites

1. Java 17+ and Spring Boot.
2. MySQL database installed and running.
3. Node.js and npm installed for React.
4. Postman for testing APIs.
5. Basic knowledge of Spring Security, JWT, React, and Axios.

Project Description

You are tasked to develop an **Income Tax Management System**. The system should allow:

- **Registering taxpayers** with details such as Name, PAN Number, and Annual Income.
- **Viewing all taxpayers**.
- **Updating taxpayer information**.
- **Deleting taxpayers**.
- **Securing the backend APIs** with JWT.
- **Frontend forms** should validate input before sending to backend using **Formik and Yup**.

Backend Requirements

1. Project Structure

```
com.incometax
│
├── config
│   ├── SecurityConfig.java
│   └── JwtAuthenticationFilter.java
│
├── controller
│   ├── AuthController.java
│   └── TaxPayerController.java
│
├── model
│   ├── TaxPayer.java
│   └── Role.java
│
├── repository
│   ├── TaxPayerRepository.java
│   └── UserRepository.java
│
├── service
│   ├── TaxPayerService.java
│   └── UserService.java
│
├── security
│   ├── JwtUtils.java
│   └── CustomUserDetailsService.java
│
└── IncomeTaxApplication.java
```

2. Functional Requirements

- **TaxPayer entity:**
    - Fields: `id`, `name`, `panNumber`, `annualIncome`
    - Validations: `name` and `panNumber` cannot be blank; `annualIncome` must be positive.
- **CRUD operations** for TaxPayer.
- **JWT-based authentication** for all endpoints except login and register.
- **Spring Data JPA** repository for database operations.

3. Testing

- Use **Postman** to test all endpoints:
    - `POST /api/taxpayers` → create a taxpayer
    - `GET /api/taxpayers` → retrieve all taxpayers
    - `GET /api/taxpayers/{id}` → retrieve by ID
    - `PUT /api/taxpayers/{id}` → update taxpayer
    - `DELETE /api/taxpayers/{id}` → delete taxpayer

Frontend Requirements

1. Project Structure

```
src/
├── api/
│   └── api.js        // Axios instance
├── components/
│   ├── TaxPayerForm.jsx
│   └── TaxPayerList.jsx
├── pages/
│   └── Dashboard.jsx
├── App.jsx
└── index.js
```

2. Functional Requirements

1. **Dashboard page** showing:
    - TaxPayer registration form
    - TaxPayer list
2. **TaxPayerForm.jsx**:
    - Fields: Name, PAN Number, Annual Income
    - Client-side validation using **Formik + Yup**
    - Submit data to backend using **Axios**
3. **TaxPayerList.jsx**:
    - Display all taxpayers in a list/table
    - Ability to delete taxpayers
    - edit taxpayers
4. **JWT Token Handling**:
    - Store JWT token in `localStorage`
    - Send token with every request automatically via Axios interceptor

3. Testing

- Open React app in browser
- Fill and submit **TaxPayerForm**
- View list of taxpayers
- Delete a taxpayer and observe changes

1. Capture Backend Screenshots (Postman)

For each CRUD operation, take screenshots in Postman:

Operation	Endpoint	Screenshot Idea
Create	POST `/api/taxpayers`	Show request body with sample data, response with newly created taxpayer
Read All	GET `/api/taxpayers`	Show list of all taxpayers returned
Read By ID	GET `/api/taxpayers/{id}`	Show single taxpayer data
Update	PUT `/api/taxpayers/{id}`	Show request with updated fields and updated response
Delete	DELETE `/api/taxpayers/{id}`	Show request sent and success response

---

2. Capture Frontend Screenshots (React App)

For the React dashboard:

Operation	Screenshot Idea
Create	Fill and submit `TaxPayerForm.jsx`, show success feedback
Read All	`TaxPayerList.jsx` showing list of taxpayers
Update	Edit a taxpayer in `TaxPayerList.jsx` and show updated info
Delete	Delete a taxpayer and show updated list

Deliverables

1. **Backend**:
    - Spring Boot project folder
    - Database schema / MySQL dump
    - Postman collection for APIs
2. **Frontend**:
    - React project folder
    - Screenshots of UI working (create, read, update, delete)
3. **Report** :

- Brief description of project
- Screenshots of Postman and UI
- Observations and issues faced

Bonus / Advanced Tasks

- Add **role-based authentication** (admin vs user).
- Add **income tax calculation** based on annual income.